

【项目总结与当前进度说明文档】

（可直接复制给另一个 ChatGPT 继续工作）

一、项目名称（当前测试环境）

Test Run Data Center

这是用于构建“个人视觉艺术数据库 / 素材分类系统”的测试场景。

二、目前数据库结构（内容层）

整个内容层独立于任何工具（Obsidian/Hugo/Flask/SQLite），易于长期维护。

Test Run Data Center/

```
images/  
  2025/  
    12/  
      *.jpg    ← 所有测试图片  
notes/  
  *.md        ← 程序自动生成的描述文件（Markdown + YAML front  
matter）  
templates/  
  index.html  ← 网页界面：图片列表 / 生成描述文件入口  
  edit.html   ← 网页界面：编辑单张图片的表单  
app.py       ← 本地 Flask Web 服务（可视化描述生成器）
```

✓ 内容层规则：

1. 图片按时间划分目录（images/YYYY/MM）
2. 每张图片对应一个 Markdown 描述文件
3. Markdown 内使用 YAML front matter 存储结构化字段
4. 工具层（Flask UI、未来的静态网页、Obsidian、AI 标签器等）均可替换
5. 内容结构保持稳定：文件名、路径、YAML 字段是核心信息源

三、目前实现的功能（通过 Flask 本地 Web 服务）

访问 `http://127.0.0.1:5000` 即可使用图形界面。

✓ 已实现功能列表

- 自动扫描 `images/` 目录获取所有图片
- 网格方式展示图片缩略图
- 提取文件名中的日期（如 `20251203_xxx.jpg` → `2025-12-03`）
- 每张图片显示“已生成 / 未生成描述文件”的状态
- 通过 Web 表单生成单张图片的 `.md` 描述文件
- YAML 自动生成以下字段：

```
id:
file_name:
path:
date:
tags:
media_type:
style_tags:
subject_tags:
composition_tags:
color_tags:
notes:
```

- Web 表单填写的字段包括：
 - `id`（默认=文件名）
 - `date`（自动解析）
 - `media_type`
 - `tags`（逗号分隔）
 - `body`（Markdown 正文）
- 描述文件自动保存在：

Test Run Data Center/notes/*.md

- 图像通过 Flask 接口本地显示，跨平台（macOS / Windows）均可运行。
-

四、Python 主程序（app.py）

位于：

Test Run Data Center/app.py

完整代码如下（可复制）

<details> <summary>点击展开 app.py 完整代码</summary>

```
from flask import Flask, render_template, request, redirect, url_for,
flash, send_from_directory
from pathlib import Path
from datetime import datetime
import os
import re
```

```
app = Flask(__name__)
app.secret_key = "change-this" # 随便一个字符串，用于 flash 消息
```

```
# 路径配置（相对项目根目录）
IMAGES_ROOT = Path("images")
NOTES_DIR = Path("notes")
```

```
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".webp"}
```

```
TEMPLATE = """---
id: {id}
file_name: {file_name}
path: {rel_path}
date: {date}
source:
  type: ""
  url: ""
  author: ""
  title: ""
{tags_block}
media_type: "{media_type}"
style_tags: []
subject_tags: []
composition_tags: []
color_tags: []
notes: ""
---
```

```
{body}
"""
```

```
def extract_date_from_filename(filename: str) -> str:
    match = re.match(r"(\d{8})", filename)
    if match:
        try:
            d = datetime.strptime(match.group(1), "%Y%m%d")
            return d.date().isoformat()
        except ValueError:
            return ""
    return ""

def list_images():
    images = []
    if not IMAGES_ROOT.exists():
        return images

    for path in IMAGES_ROOT.rglob("*"):
        if path.is_file() and path.suffix.lower() in IMAGE_EXTS:
            rel_path = path.as_posix()
            rel_under_images =
path.relative_to(IMAGES_ROOT).as_posix()
            stem = path.stem
            date = extract_date_from_filename(path.name)
            md_path = NOTES_DIR / f"{stem}.md"
            has_md = md_path.exists()
            images.append({
                "file_name": path.name,
                "rel_path": rel_path,
                "rel_under_images": rel_under_images,
                "stem": stem,
                "date": date,
                "has_md": has_md,
            })

    images.sort(key=lambda x: x["rel_path"])
    return images
```

```
@app.route("/")
```

```

def index():
    images = list_images()
    return render_template("index.html", images=images)

@app.route("/edit")
def edit():
    rel_path = request.args.get("image")
    if not rel_path:
        return redirect(url_for("index"))

    p = Path(rel_path)
    default_id = p.stem
    default_date = extract_date_from_filename(p.name)

    return render_template(
        "edit.html",
        rel_path=rel_path,
        file_name=p.name,
        default_id=default_id,
        default_date=default_date
    )

@app.route("/save", methods=["POST"])
def save():
    rel_path = request.form["rel_path"]
    file_name = request.form["file_name"]

    id_val = (request.form.get("id") or "").strip() or
    Path(file_name).stem
    date_val = (request.form.get("date") or "").strip()
    media_type = (request.form.get("media_type") or "").strip()
    tags_str = (request.form.get("tags") or "").strip()
    body = request.form.get("body") or ""

    if date_val:
        date_yaml = f"\{date_val}\n"
    else:
        date_yaml = '""'

    tags_list = [t.strip() for t in tags_str.split(",") if t.strip()]
    if tags_list:

```

```

        tags_block = "tags:\n" + "\n".join(f" - {t}" for t in
tags_list)
    else:
        tags_block = "tags: []"

    content = TEMPLATE.format(
        id=id_val,
        file_name=file_name,
        rel_path=rel_path,
        date=date_yaml,
        tags_block=tags_block,
        media_type=media_type,
        body=body
    )

    NOTES_DIR.mkdir(exist_ok=True)
    stem = Path(file_name).stem
    md_path = NOTES_DIR / f"{stem}.md"

    with open(md_path, "w", encoding="utf-8") as f:
        f.write(content)

    flash(f"已生成描述文件: {md_path.as_posix()}")
    return redirect(url_for("index"))

@app.route("/img/<path:filename>")
def serve_image(filename):
    return send_from_directory(IMAGES_ROOT, filename)

if __name__ == "__main__":
    app.run(debug=True)

```

</details>

五、前端模板文件（HTML）

放在：

Test Run Data Center/templates/

1 index.html (图片列表页)

<details> <summary>点击展开 index.html 代码</summary>

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8" />
  <title>图像描述文件生成器</title>
  <style>
    body { font-family: -apple-system, BlinkMacSystemFont, sans-serif; padding: 20px; }
    .grid { display: flex; flex-wrap: wrap; gap: 16px; }
    .card {
      border: 1px solid #ccc;
      border-radius: 8px;
      padding: 8px;
      width: 180px;
      box-sizing: border-box;
    }
    .thumb {
      width: 100%;
      height: 140px;
      object-fit: contain;
      border-radius: 4px;
      background: #f5f5f5;
    }
    .meta { font-size: 12px; margin-top: 4px; }
    .badge { padding: 2px 6px; background: #eee; border-radius: 8px; font-size: 10px; }
    .btn { padding: 4px 8px; border: 1px solid #333; border-radius: 4px; text-decoration: none; }
  </style>
</head>
<body>
  <h1>图像描述文件生成器</h1>

  {% with messages = get_flashed_messages() %}
  {% if messages %}
    <div class="flash">
      {% for msg in messages %}
        <div>{{ msg }}</div>
      {% endfor %}
    </div>
  {% endif %}
  </body>
</html>
```

```

        </div>
    {% endif %}
{% endwith %}

<div class="grid">
    {% for img in images %}
        <div class="card">
            

            <div class="meta">
                <div>{{ img.file_name }}</div>
                {% if img.date %}
                    <div>日期: {{ img.date }}</div>
                {% endif %}
                <div class="badge">{{ ' 已生成描述' if img.has_md else ' 未生
成' }}</div>
            </div>

            <a class="btn" href="{{ url_for('edit',
image=img.rel_path) }}">
                {{ ' 编辑描述' if img.has_md else ' 生成描述文件' }}
            </a>
        </div>
    {% endfor %}
</div>
</body>
</html>
</details>

```

2 edit.html (描述文件编辑页)

<details> <summary>点击展开 edit.html 代码</summary>

```

<!doctype html>
<html lang="zh-CN">
<head>
    <meta charset="utf-8" />
    <title>编辑描述文件</title>

```



```

<style>
  body { font-family: -apple-system, BlinkMacSystemFont, sans-
serif; padding: 20px; }
  label { display: block; margin-top: 10px; }
  input[type="text"], textarea {
    width: 100%; padding: 6px; font-size: 14px; margin-top: 4px;
  }
  textarea { min-height: 140px; }
  .thumb { width: 240px; height: 240px; object-fit: contain;
border: 1px solid #ccc; }
</style>
</head>
<body>
  <h1>编辑描述文件</h1>

  

  <form method="post" action="{{ url_for('save') }}">
    <input type="hidden" name="rel_path" value="{{ rel_path }}">
    <input type="hidden" name="file_name" value="{{ file_name }}">

    <label>ID:
      <input type="text" name="id" value="{{ default_id }}">
    </label>

    <label>日期:
      <input type="text" name="date" value="{{ default_date }}">
    </label>

    <label>媒介类型:
      <input type="text" name="media_type">
    </label>

    <label>标签（逗号分隔）:
      <input type="text" name="tags">
    </label>

    <label>正文内容:
      <textarea name="body"></textarea>
    </label>

    <button type="submit">保存描述文件</button>

```

```
</form>

<p><a href="{{ url_for('index') }}">← 返回</a></p>
</body>
</html>
</details>
```

六、程序运行方式

① 创建虚拟环境并安装 Flask

macOS:

```
cd "Test Run Data Center"
python3 -m venv venv
source venv/bin/activate
pip install flask
```

Windows:

```
cd "Test Run Data Center"
python -m venv venv
venv\Scripts\activate
pip install flask
```

② 运行程序

```
python app.py
```

打开浏览器访问:

```
http://127.0.0.1:5000
```

即可看到可视化界面。

七、未来扩展（可交给另一个 ChatGPT 开发）

你可以让它继续开发：

- 新字段（自动补全 / 下拉列表 / 标签体系）
 - 自动 AI 标签（调用模型分析图片）
 - 批量生成描述文件
 - 更美观的 UI（卡片阴影、瀑布流布局、搜索过滤）
 - 导出 JSON / SQLite 作为内容层索引
 - 生成静态网站（Hugo/Eleventy）
 - 添加全局设置文件（config.yaml）
 - 文件重命名工具（确保 ID 稳定性）
-

✓ 完整总结（一句话）

你现在拥有一个：

采用“内容层独立设计”的个人视觉艺术数据库，
使用 Flask 提供本地可视化界面，自动为每张图片生成 Markdown + YAML 描述文件，
数据库结构稳定、可扩展、跨平台可运行。